

თბილისის თავისუფალი უნივერსიტეტი

კომპიუტერული მეცნიერებების, მათემატიკისა და ინჟინერიის სკოლა (MACS[E])
ელექტრო და კომპიუტერული ინჟინერიის ბაკალავრიატის პროგრამა



ლაბორატორიული პროექტის ანგარიში

თვითბალანსირებადი რობოტი

(Self-Balancing Robot)

ავტორები:

ნონია ჯავახიშვილი

საბა კვეტენაძე

თბილისი, 2026

FREE UNIVERSITY OF TBILISI

School of Mathematics, Computer Science and Engineering (MACS[E])

Electronic and Computer Engineering Undergraduate Program



LABORATORY PROJECT REPORT

Self-Balancing Robot Design and Implementation

PREPARED BY:

Nonia Javakhishvili

Saba Kvetenadze

Tbilisi, 2026

ანოტაცია

ამ პროექტის ფარგლებში შეიქმნა მაღალი წარმადობის ESP32 მიკროკონტროლერზე დაფუძნებული ორბორბლიანი, თვითბალანსირებადი რობოტი. პროექტის მიზანი იყო ისეთი სისტემის აგება, რომელიც შებრუნებული (ინვერსიული) ქანქარის ფიზიკურ პრინციპზე დაყრდნობით, რეალურ დროში შეძლებდა ვერტიკალური წონასწორობის შენარჩუნებას.

სისტემა იყენებს MPU6050 ინერციულ საზომ მოდულს (IMU) დახრის კუთხისა და კუთხური სიჩქარის მუდმივი მონიტორინგისთვის. სენსორის ხმაურის შესამცირებლად და ზუსტი ორიენტაციის დასადგენად გამოყენებულია კომპლემენტარული ფილტრი. მიღებულ მონაცემებს ამუშავებს ESP32 (FreeRTOS არქიტექტურით), რომელიც PID კონტროლერის ალგორითმის მეშვეობით ადგენს საჭირო სიმძლავრეს და წონასწორობის აღსადგენად მართავს L298N დრაივერსა და TT DC ძრავებს.

დამატებით, რობოტში ინტეგრირებულია Wi-Fi Web სერვერი, რომელიც უზრუნველყოფს ცოცხალ (Live) ტელემეტრიას, ბატარეის მონიტორინგს, მოძრაობის კონტროლსა და PID კოეფიციენტების დისტანციურ რეგულირებას. სისტემის საიმედოობისთვის კოდში გაწერილია უსაფრთხოების მექანიზმები, როგორიცაა I2C კომუნიკაციის გაჭედვის აღმოფხვრა და კავშირის დაკარგვისას ძრავების გაჩერება (Dead-man switch). პროექტმა წარმატებით აჩვენა აპარატურული და კომპლექსური პროგრამული კომპონენტების ინტეგრაციის შესაძლებლობა დინამიკური სტაბილიზაციის მქონე რობოტექნიკურ სისტემებში.

Abstract

This project details the design and construction of a high-performance, two-wheeled Self-Balancing Robot governed by the ESP32 microcontroller. Operating on the physical paradigm of an inverted pendulum, the system aims to preserve its vertical equilibrium actively in real time. To acquire orientation metrics, an MPU6050 Inertial Measurement Unit (IMU) samples axial acceleration and angular velocity. A complementary filtering technique is deployed to reconcile high-frequency noise and long-term sensor drift.

The processed orientation error feeds into a discrete-time Proportional-Integral-Derivative (PID) control loop running on a dedicated FreeRTOS core. The resulting control output is translated into PWM signals directed to an L298N dual H-bridge motor driver, regulating twin TT DC gear motors. Furthermore, the system hosts a multi-endpoint Wi-Fi Web Server acting as a remote control and tuning dashboard. This interface provides live telemetry, battery voltage monitoring, and dynamic modification of PID constants.

To ensure robust operation, software-level safety protocols were engineered, including an automatic I2C bus recovery mechanism for sensor lockups and a dead-man's switch to halt motors upon connection loss. The completed system successfully demonstrates advanced embedded systems programming, RTOS integration, and physical stabilization.

Table of Contents

1. Introduction	4
2. Technical Overview of the Project	4
2.1 Hardware Architecture	4
3. Software Architecture & Control Logic	5
3.1 Dual-Core RTOS Implementation	5
3.2 Sensor Fusion (Complementary Filter)	5
3.3 PID Control & Deadband Compensation	6
3.4 Web Server & Live Telemetry	6
3.5 Active Safety Mechanisms	7
4. Experience and Knowledge Gained	7
5. Conclusion	8

1. Introduction

In the field of robotics and control systems engineering, the two-wheeled self-balancing robot represents a benchmark challenge. The system mimics an **inverted pendulum**—a classic mechanical configuration where the center of mass resides above its pivot axis. Left unassisted, this arrangement is inherently unstable and governed by gravitational forces that cause immediate falling.

The objective of this laboratory project was to engineer an advanced self-balancing robot using the 32-bit dual-core ESP32 microcontroller. Unlike standard Arduino-based robots, this project leverages the ESP32's capabilities to simultaneously run high-speed physical stabilization algorithms and a concurrent Wi-Fi web server. This allows the robot not only to balance itself but also to stream live telemetry, monitor battery health, and accept remote driving commands via an interactive HTML/JS dashboard.

2. Technical Overview of the Project

2.1 Hardware Architecture

The structural and operational execution of the robot relies on a modular set of components:

Component Name	Functional Role in the Subsystem
ESP32 Development Board	Acts as the central processing unit utilizing FreeRTOS. Handles sensor parsing, PID math, and hosts the Wi-Fi web server simultaneously.
MPU6050 IMU	6-axis sensor containing a 3-axis accelerometer and 3-axis gyroscope. Communicates kinematic data over the I2C protocol at 400kHz.
L298N Motor Driver	A dual H-bridge driver that translates logic-level PWM signals from the ESP32 into high-current outputs required by the motors.
TT DC Gear Motors	Provide the mechanical thrust required to move the robot forward or backward to continuously adjust the base under the center of mass.
Battery Voltage Divider	An analog sensor setup routed to ADC Pin 34 to allow the microcontroller to measure the remaining 2S Li-ion battery voltage actively.

3. Software Architecture & Control Logic

3.1 Dual-Core RTOS Implementation

The most significant technical achievement of the software is the use of the FreeRTOS framework to divide processing tasks across the ESP32's dual-core processor. This ensures that network latency does not interfere with physical balance:

- **Core 1 (BalanceTask):** Pinned exclusively to real-time control. It reads the MPU6050, computes the PID loop, and updates motor PWM signals every 5 milliseconds (200Hz).
- **Core 0 (TelemetryTask):** Handles non-time-critical asynchronous tasks such as maintaining the Wi-Fi connection, serving HTTP client requests, and polling the analog battery pin.

Data synchronization between the cores (such as passing new PID values from the Web UI to the balance loop) is safely managed using an RTOS Mutex (`xDataMutex`) to prevent race conditions.

3.2 Sensor Fusion (Complementary Filter)

Maintaining balance relies on calculating an accurate pitch angle. The accelerometer is prone to mechanical noise, while the gyroscope drifts over time due to numerical integration. To solve this, a **Complementary Filter** was hardcoded into the loop:

$$\text{robotAngle} = \alpha \cdot (\text{robotAngle} + \text{gyroRate} \cdot dt) + (1 - \alpha) \cdot \text{accAngle}$$

Using a tuning coefficient of $\alpha = 0.98$, the formula heavily trusts the integrated gyroscope data for short-term precision (98%) while continuously pulling the angle toward the absolute accelerometer reading (2%) to cancel out drift over time.

3.3 PID Control & Deadband Compensation

The calculated angle is processed by a Proportional-Integral-Derivative (PID) algorithm. The terms (K_p , K_i , K_d) generate a corrective motor output to accelerate the wheels toward the direction of the fall.

An observed physical limitation was the "deadband" of the TT Gear Motors—a low PWM region where voltage is applied but insufficient to overcome internal mechanical friction. To solve this, a programmatic **Progressive Gearbox Friction Deadzone** was added. If the PID requests any movement (>0.1), the software automatically injects an offset base voltage (e.g., ± 45 PWM) to instantly break the static friction, ensuring the robot reacts immediately to microscopic tilt errors.

3.4 Web Server & Live Telemetry

Instead of hardcoding tuning variables, the ESP32 hosts a built-in asynchronous HTTP server accessible via IP address. The web server exposes several RESTful API endpoints:

- `/telemetry`: Streams a JSON payload containing the current robot angle, battery voltage, and Wi-Fi RSSI strength to the browser dashboard.
- `/set_pid`: Receives real-time parameter updates for Kp, Ki, Kd, and the target balance angle, applying them instantly via Mutex.
- `/control`: Receives directional offsets (move and turn) from the HTML slider interface, enabling the user to drive the robot remotely without destabilizing the core PID loop.

3.5 Active Safety Mechanisms

Robotics deployed in real-world environments require fault tolerance. The code introduces several innovative safety checks:

1. I2C Freeze Recovery: MPU6050 modules can occasionally lock up due to electrical noise from motors, causing the I2C bus to freeze and the robot to crash. The code actively monitors for duplicate raw sensor values. If the values repeat for 50 consecutive cycles (250ms), the system aggressively shuts down the I2C bus, resets the wire, re-initializes the MPU6050, and reapplies gyro offsets on the fly.

2. RC Dead-Man's Switch: When driving via the Web Interface, network packets can be lost. To prevent a "ghost joystick" runaway scenario, the code utilizes a `millis()` timer. If a control packet is not received for over 1000 milliseconds, the directional commands are forced to zero, bringing the robot to a safe, stationary balance.

4. Experience and Knowledge Gained

This project elevated fundamental electronics skills into the domain of advanced embedded systems architecture. Key technical takeaways included:

- **RTOS Resource Management:** Learning how to utilize FreeRTOS to pin specific tasks to specific CPU cores and using Mutexes to avoid memory corruption between parallel threads.
- **Electrical Fault Mitigation:** Diagnosing and writing software workarounds for hardware-level flaws, specifically the I2C freezing issues and the deadband voltage of cheap gear motors.

- **IoT Integration:** Establishing a reliable Wi-Fi stack and designing an HTTP interface capable of bidirectional, low-latency communication with a custom HTML/JS frontend.

5. Conclusion

This project demonstrated the successful implementation of a highly sophisticated, ESP32-based two-wheeled self-balancing robot. By leveraging a Complementary Filter for accurate sensor fusion and a rigid FreeRTOS task structure, the system achieves remarkable physical stability.

The integration of a concurrent Wi-Fi web server transformed the robot from a simple balancing platform into an interactive, remote-controlled smart device. The addition of robust safety protocols, such as I2C bus recovery and network timeout protections, highlights a professional approach to embedded software design. In conclusion, the project effectively merged classical control theory (PID), modern IoT web development, and advanced RTOS multitasking into a single, highly functional robotic platform.